

Magic Pro DAW — Project Deep Dive Report

Codebase scan, architecture map, current implementation status, and recommendations

Workspace: C:\personal\daw

Generated: 2026-06-02 09:55

Source report: PROJECT_REPORT.md

1. Executive Summary

Magic Pro DAW is a feature-rich, in-progress browser-based Digital Audio Workstation built primarily on Next.js 14 + TypeScript, with a real-time audio engine written against the Web Audio API / AudioWorklets and a Rust -> WebAssembly DSP core for performance-critical processing.

The codebase is large and ambitious: roughly 559 source files spanning around 80,740 lines of code (TS/TSX/JS/JSX/Rust, excluding node_modules, .next, .git, and vendored content). The project follows a clear separation between UI (app/, components/), state (store/), engine (engine/), persistence (engine/persistence/, lib/db.ts, prisma/), and the WASM DSP layer (wasm/dsp-core/).

The README describes a narrower scope (single-user, multi-track, mixer, MIDI) than the resolved implementation plan (implementation_plan.md.resolved) which targets a full SoundForge Studio platform: auth, cloud storage, real-time CRDT collaboration, AI music assistant, and a plugin system. The actual code appears to be a hybrid — most of the multi-track/audio/MIDI scope is implemented, while collaboration, AI, and S3 storage are scaffolded but not deeply wired.

2. Tech Stack (Observed)

Layer	Technology	Notes
Framework	Next.js 14.1.0 (App Router)	app/ directory routing
Language	TypeScript 5 (strict: true)	Path alias @/* -> root
UI	React 18 + Tailwind CSS 3 + lucide-react icons	
State	Zustand 4.5 with immer 10	7 stores in store/
Auth	NextAuth.js 4.24	Credentials provider + bcryptjs
ORM/DB	Prisma 5.22 + PostgreSQL (SQLite fallback)	Schema in prisma/schema.prisma
Optional services	Supabase (@supabase/supabase-js), Firebase	Both installed but lightly used
Audio	Web Audio API + AudioWorklets	Custom scheduler (25-50ms lookahead)
DSP	Rust -> WebAssembly (wasm-bindgen)	EQ + Compressor processors
Testing	Jest 30 + ts-jest	jest.config.js present
Lint	ESLint 8 + eslint-config-next	
Build helpers	autoprefixer, postcss	

Cross-Origin Isolation is configured in next.config.js (COOP/COEP/CORP headers) — required for SharedArrayBuffer and worklet-based shared-memory transport.

3. Repository Layout

C:\personal\daw\
■■■ app/ # Next.js App Router (21 TS/TSX files)
■■■ components/ # React UI (117 files; largest dir)
■■■ engine/ # Audio engine & DSP (180 files; largest)
■■■ store/ # Zustand stores (7 files)
■■■ lib/ # Shared utilities (13 files)
■■■ models/ # Domain types (4 files)
■■■ prisma/ # Schema + seed (2 files)
■■■ templates/ # Project templates (7 files)

```

■■■ wasm/dsp-core/ # Rust WASM DSP (4 files)
■■■ hooks/ # React hooks (3 files)
■■■ public/worklets/ # AudioWorklet processors
■■■ tests/, scripts/, docs/, scratch/, tmp/, types/ # utility / doc folders
■■■ Aider-AI-aider-a4be6cc/ # Vendored Aider source
■■■ legacy/ magic-pro-modules/ vst/ sound sample/ # legacy/asset dirs
■■■ implementation_plan.md.resolved # Full architecture plan
■■■ README.md
■■■ package.json

```

- engine/: 180 files — the bulk of the system
- components/: 117 files — UI
- app/: 21 files — routes + API
- store/: 7 stores
- wasm/: 4 Rust files
- lib/, models/, prisma/, templates/, hooks/: supporting modules

4. Architecture by Subsystem

4.1 Data Model

PostgreSQL schema (provider configurable via DATABASE_URL; defaults to file:./dev.db per README):

- User — id, email, passwordHash, name; owns many Projects.
- Project — id, userId, name, tempo, timeSignature, keySignature, projectFormat, surroundFormat, spatialAudioMode, stateJson, shareId, isPublic, lastOpenedAt.
- Track — id, projectId, name, type (audio|midi|drum|aux|bus), volume, pan, muted, soloed, color, orderIndex; has clips, plugins, automation, sends.
- Clip — id, trackId, type (audio|midi), start, duration, name, color, fileId; has notes.
- Note — id, clipId, pitch, velocity, start, duration.
- Automation / AutomationPoint — per-track parameter curves.
- Plugin — per-track insert slot + settingsJson.
- Bus / Send — send routing.

Domain TypeScript types live in models/: Track.ts, Clip.ts, Project.ts, Articulation.ts. Plus models/rendering/ and models/runtime/.

4.2 State Management — store/

Seven Zustand stores, all client-side and imported heavily by components.

Store	Concern
projectStore.ts	Core project state — tracks, clips, transport, history, automation, environment, alternatives, settings (4,802 lines — the single largest file)
midiStore.ts	MIDI recording, input, editing state
mixerStore.ts	Mixer layout, sends, mute/solo state
automationStore.ts	Automation lanes, points, gestures
clipEditingStore.ts	Clip edit selections, tool state
onboardingStore.ts	First-run / tutorial flow
tutorialStore.ts	Tutorial steps

The projectStore couples deeply with the audio engine via `@/engine/AudioEngineAdapter` and includes `serializeStoreState/deserializeState/saveToIndexedDB/loadFromIndexedDB` from `engine/persistence/projectPersistence.ts`.

4.3 Audio Engine — engine/ (180 files)

Heavily modular. Key submodules:

- `engine/audioEngine/` — Core runtime: `audioContext.ts`, `scheduler.ts` (606 lines, 25-50ms lookahead), `recordingEngine.ts`, `routingEngine.ts`, `masterBus.ts`, `metronome.ts`, `bufferCache.ts`, `bounceEngine.ts` (offline render), `clipPlaybackController.ts`, `clipDSP.ts`, `channelStrip.ts`; `dsp/{channelStrip.processor.ts, synth.processor.ts, timeStretch.processor.ts}` — worklet processors; `__tests__/{routing.test.ts, scheduler.test.ts}` — Jest unit tests; `README.md`, `README_MIXER.md` — extensive inline docs.
- `engine/audioRecording/` — `recorder.ts`, `inputManager.ts`, `bufferManager.ts`, `wavEncoder.ts`, `liveWaveform.ts`, `waveformAnalyzer.ts`.
- `engine/midi/` — `midiScheduler.ts`, `MidiRenderer.ts`, `MidiQuantizer.ts`, `MidiTools.ts`, `MidiHumanizer.ts`, `TransportTimeline.ts`, `midiTransforms.ts`, `quantization.ts`, `MidiStateResolver.ts`, `MidiPlaybackInvalidation.ts`, `MidiCommands.ts`, `midiEditor.ts`.
- `engine/timeline/` — Canvas timeline renderer, clip editing, slip editing, crossfade engine, history manager, ghost clips, waveform cache; plus `CLIP_EDITING_ARCHITECTURE.md`.
- `engine/automation/` — A substantial system: curves, interpolation, parameter binding, compiler, spatial cache, runtime (`SampleAccurateModulator`, `AutomationLookahead`, `ParameterStreamRuntime`), indexing, gesture engine, overlay, lane/bezier rendering.
- `engine/rendering/` — `RenderGraph`, `RenderPass`, `cache/SpatialCache`, dirty region manager, frame profiler; WebGL pipeline: `WebGLRenderer.ts`, `WebGLBatcher.ts`, `shaders` (`CurveShader`, `GridShader`, `NoteShader`).
- `engine/navigation/` — Gesture interpreter, viewport transaction, spatial coordinate system, playhead follow, velocity integrator, constraint pass, automation viewport client, `useNavigation.ts` hook.
- `engine/editor/` — `EditorCore.ts`, `InteractionManager.ts`, `SelectionManager.ts`, `SnapEngine.ts`, `ToolManager.ts`; tools: `SelectTool`, `MarqueeTool`, `SplitTool`, `DrawTool`.
- `engine/instruments/` — `synthEngine.ts`, `samplerEngine.ts`, `multiSamplerEngine.ts`, `drumMachine.ts`, `instrumentFactory.ts`, `instrumentRegistry.ts`, `midilIntegration.ts`.
- `engine/persistence/` — `projectPersistence.ts`, `engineRebuilder.ts`, `audioFileStore.ts`, `autosave.ts`, `migration.ts`.
- `engine/filesystem/` — `projectManager.ts`, `projectSerializer.ts`, `assetManager.ts`, `autosaveManager.ts`, `importManager.ts`, `exportManager.ts`, `indexedDBAdapter.ts`.
- `engine/export/` — `OfflineRenderer.ts`, `wavEncoder.ts`.
- `engine/effects/plugins/` — `compressorPlugin.ts`, `eqPlugin.ts`.
- `engine/performance/` — `audioGraphManager.ts`, `nodePool.ts`, `renderOptimizer.ts`.
- `engine/collaboration/` — `ProjectCRDTSync.ts`, `crdt/CRDTProvider.ts` (scaffold only).

- engine/dsp/, engine/gpu/, engine/soundLibrary/, engine/runtime/ — additional supporting modules.
- Public worklets (public/worklets/): DSPWorkletProcessor.js, synth-processor.js.

4.4 UI Layer

Routes (app/):

- app/page.tsx — landing.
- app/welcome/, app/login/, app/signup/.
- app/dashboard/ — project list.
- app/project/[projectId]/page.tsx — main DAW workspace (396 lines, composes ~20 components).
- app/p/[shareId]/ — public share view.
- app/account/.
- app/debug-audio/, app/debug/runtime/ — dev/diagnostic pages.
- middleware.ts — root middleware.

API routes (app/api/):

- auth/[...nextauth]/route.ts + auth/signup/route.ts.
- account/update/route.ts.
- project/save/route.ts, project/[id]/route.ts, project/[id]/share/route.ts.
- projects/route.ts (list).
- public/[shareId]/route.ts (read-only share).

Components (components/, 117 files) are heavily Logic-Pro-inspired. Highlights:

- Core DAW surface: TransportBar, Toolbar, TrackList, Timeline, TimelineCanvas, TimelineWithClipEditing, Mixer, PianoRoll, Inspector, SmartControls, LibraryPanel, LoopBrowser, Browsers.
- MIDI: midi/{MidiGrid, MidiCanvasGrid, MidiNote, MidiNoteCanvas, PianoRoll, PianoRollTools, PianoKeyboard, VelocityLane}.tsx.
- Mixer: mixer/{ChannelStrip, Meter, Mixer, MixerChannel, MixerFader, MixerMeter, SendControls}.tsx.
- Plugins: plugins/{WasmCompressorUI, WasmEQUI}.tsx, Compressor.tsx, ChannelEQ.tsx, TapeDelay.tsx, ChromaVerb.tsx.
- Automation: automation/{AutomationCurve, AutomationEditor, AutomationLane, AutomationPoint}.tsx, AutomationRuntimeOverlay.
- Clip editing: Clip.tsx, ClipHandles.tsx, ClipGainHandle.tsx, ClipContextMenu.tsx, CrossfadeHandle.tsx.
- Layout: layout/{DAWWorkspace, DAWLayoutExample, TopTransport, HorizontalSplitView, HorizontalResizeHandle}.tsx.
- Dialogs: BounceDialog, BounceAllTracksDialog, BounceRegionsDialog, BounceTrackDialog, ExportDialog, ImportProjectDialog, ProjectManager, NewTrackDialog, ShareDialog, ShareModal, PreferencesDialog, SaveDialog, NoteRepeatDialog, SpotEraseDialog, DrumReplacementDialog, TrackHeaderConfigDialog, ArticulationSetEditor, CreateNewTrackUsingDialog, ColorPalette, IconBrowser, SearchAndSelectDialog.
- Specialized: LiveLoopsGrid, LiveRecordingWaveform, RecordingInputMeter, TrackLevelMeter, VerticalMeter, HorizontalMeter, MasterOutput, GlobalKeyHandler, GlobalTracks, NotePad, OnboardingOverlay, QuickHelpWindow, QuickSoundBrowser, SelectionBasedProcessing, StepInputKeyboard, TracksAreaMenuBar, ViewControlBar, VirtualKeyboard, WaveformCanvas,

WaveformSVG, AppMenuBar, ListEditors, ErrorBoundary, Toast.

- Adapters: adapters/ProjectPianoRollAdapter.tsx.
- Filesystem: filesystem/{ExportDialog, ImportDialog, ProjectBrowser}.tsx.

4.5 WASM DSP Core — wasm/dsp-core/

- Cargo.toml — magic-dsp-core 0.1.0, cdylib output, wasm-bindgen 0.2, LTO + opt-level 3.
- src/lib.rs — entrypoint.
- src/processors/eq.rs, compressor.rs, mod.rs — implemented in Rust.
- build.ps1 — Windows build script.
- A rust/ folder at the project root contains 9 additional Rust files (parallel/related crate).

4.6 Templates — templates/

Starter project templates registered in templates/index.ts: lofi.ts, hip-hop.ts, piano.ts, edm.ts, podcast.ts; types.ts defines ProjectTemplate, TemplateTrackDef, TemplateClipDef.

4.7 Hooks — hooks/

useErrorHandler.ts, useFullscreen.ts, useInstruments.ts.

4.8 Other

- Aider-AI-aider-a4be6cc/ — vendored Aider source (15 files).
- legacy/ — 7 leftover files from earlier iterations.
- magic-pro-modules/ — 3 files, likely a stale module scratchpad.
- vst/, sound sample/, scratch/, tmp/ — asset/scratch directories (no code).
- types/next-auth.d.ts — NextAuth type augmentation.
- docs/ — Magic_Pro_Architecture_Report.pdf + .docx + build scripts.

5. Authentication and Persistence

- Auth: NextAuth credentials provider (lib/auth.ts), bcryptjs hashing, signup API at app/api/auth/signup/route.ts. Routes are protected via middleware.ts.
- DB: Prisma + PostgreSQL (prisma/schema.prisma); prisma/seed.js populates sample data.
- Local cache: IndexedDB via engine/persistence/ (audioFileStore.ts, projectPersistence.ts, engineRebuilder.ts). engine/filesystem/indexedDBAdapter.ts wraps raw IDB.
- Sharing: shareId field on Project model; app/api/project/[id]/share/route.ts and app/p/[shareId]/page.tsx provide a public read-only view (app/api/public/[shareId]/route.ts).
- Autosave: engine/persistence/autosave.ts, engine/filesystem/autosaveManager.ts.

6. Implemented vs Planned

The implementation_plan.md.resolved (SoundForge Studio) is a much broader plan than the README. Mapping the plan to the code:

Planned capability	Code status
Multi-track timeline, mixer, transport	Implemented (deep)
WebGL rendering	Implemented (engine/rendering/webgl/*)
AudioWorklet DSP	Implemented (engine/audioEngine/dsp/*, public/worklets/*)
Automation lanes	Implemented extensively (compiler, runtime, spatial cache)
MIDI piano roll	Implemented (components/midi/*, engine/midi/*)
Recording + WAV export	Implemented (engine/audioRecording/*, engine/export/*)
Plugin host (EQ, Compressor, etc.)	Implemented in JS + WasmCompressorUI/WasmEQUI
Offline bounce	Implemented (bounceEngine.ts, OfflineRenderer.ts)
Plugin registry / third-party WASM sandbox	Partial (engine/plugins/PluginAPI.ts, PluginRegistry.ts)
Real-time CRDT collaboration (Yjs/Socket.IO)	Stub only (engine/collaboration/* — CRDTProvider.ts, ProjectCRDTSync.ts); no server WebSocket layer present
S3 presigned upload	Not present in code (Supabase/Firebase SDKs installed but unused for storage)
AI music assistant (chords, melody, stems)	Not present (no /api/ai/* routes)
Stripe / subscription tiers	Not present
Stem separation, auto-mix	Not present
PWA	Not present

Conclusion: The project has reached a mature single-user DAW state; the cloud/collab/AI side of the plan is largely aspirational.

7. Notable Engineering Choices

- Two parallel AudioEngines exist: lib/audioEngine.ts (192 lines) — the original, simple class; engine/audioEngine/* (the new, modular stack) and engine/AudioEngineAdapter.ts — the singleton bridge. This is a common migration pattern but worth consolidating.
- Project store is huge (projectStore.ts = 4,802 lines) — likely a candidate for splitting into multiple slices.
- Heavy modularization in engine/automation/, engine/rendering/, engine/navigation/ — supports the spatial-aware, sample-accurate, gesture-driven design described in the architecture PDF.
- Cross-Origin Isolation headers are set globally in next.config.js (COOP=same-origin, COEP=require-corp, CORP=cross-origin) to enable SharedArrayBuffer for the worklet transport (engine/dsp/memory/SharedTransportBuffer.ts).
- Template + persistence + rebuilder pattern: engine/persistence/engineRebuilder.ts rebuilds the audio graph from serialized state — a clean separation between state and audio runtime.
- Tests present for routing and scheduler in engine/audioEngine/__tests__/. No tests in components/, app/, or store/.
- Substantial inline documentation: docs/Magic_Pro_Architecture_Report.pdf/docx, engine/audioEngine/README.md, engine/audioEngine/README_MIXER.md, engine/midi/README_MIDI.md, engine/timeline/CLIP_EDITING_ARCHITECTURE.md, engine/filesystem/README_PROJECT.md.

8. Build / Run / Test

From package.json:

```

npm install
npx prisma generate
npx prisma db push # uses DATABASE_URL (Postgres or SQLite)
npx prisma db seed # node prisma/seed.js
npm run dev # next dev
npm run build # next build
npm run lint # eslint

```

Jest is configured (jest.config.js); run with npx jest. WASM DSP build: wasm\dsp-core\build.ps1 (PowerShell).

9. Risks and Observations

- tsconfig.tsbuildinfo and stale tsc-errors.txt, ts_errors.txt, typescript_errors.log, output.txt in root — indicate ongoing TS cleanup; not a clean repo state.
- Dead/legacy directories (legacy/, magic-pro-modules/, stray src/, sound sample/, vst/, tmp/, scratch/) inflate the surface area and should be moved to archive/ or removed.
- README.md describes a v0.1 scope; implementation_plan.md.resolved describes a v1.0 vision. Update the README to reflect actual scope, or decide which plan is the target.
- No tests for components or stores — risk surface is large.
- Large projectStore.ts is the biggest single file (4,802 lines) and tightly couples UI, persistence, and engine — refactor candidate.
- No real-time backend present — CRDTProvider.ts and ProjectCRDTSync.ts exist but there is no Socket.IO/Yjs server.
- No CI configuration files (no .github/, no Dockerfile, no vercel.json workflow beyond the basic Vercel config).
- .env is committed? .env is listed alongside .env.example and .gitignore exists — confirm .env is in .gitignore and not leaked.

10. Quick Stats

Metric	Value
Source files (TS/TSX/JS/JSX/RS/Prisma)	559
Total LOC (source)	~80,740
Largest source file	store/projectStore.ts (4,802 lines)
Largest module by file count	engine/ (180 files)
Largest module by file count (UI)	components/ (117 files)
Routes (app/)	21
API endpoints	~7
Zustand stores	7
Project templates	5 (lofi, hiphop, piano, EDM, podcast)
WASM DSP processors (Rust)	2 (eq, compressor)
Public worklets	2 (DSP, synth)

11. Suggested Next Steps

1. Triage the repository — remove/move legacy/, magic-pro-modules/, tmp/, scratch/, stray src/, asset folders, and stale log files.
2. Split projectStore.ts along domain boundaries (transport, tracks, clips, automation, persistence, environment, alternatives).
3. Resolve the dual engine — pick engine/audioEngine/* + AudioEngineAdapter as canonical, retire lib/audioEngine.ts.
4. Add component and store tests (at least smoke tests for stores and the DAW workspace page).
5. Update README to match actual scope and clearly mark the roadmap items (collab, AI, S3) as future.
6. CI pipeline — typecheck (tsc --noEmit), lint (next lint), and jest on PRs.
7. Decide the WASM strategy — the rust/ folder at root and wasm/dsp-core/ look like overlapping starts; consolidate.